

“MULTITHREADED LINUX APPLICATION USING POSIX THREADS AND MUTEXES”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

**U. MOKSHA
JAMUNA
KEERTHI**

**25200031613
2520030408
2520030416**

Under the Esteemed Guidance of

K.HEMA

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“MULTITHREADED LINUX APPLICATION USING POSIX THREADS AND MUTEXES”** is a record of Bonafide work of **JAMUNA - 2520030408** , **U.MOKSHA - 2500031613** , **KEERTHI- 2520030416** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

U. MOKSHA 2500031613

JAMUNA 2520030408

KEERTHI 2520030416

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled “**Multithreaded Linux Application using Posix and Mutexes**” is a bonafide work done and submitted by **U.Moksha - 2500031613, Jamuna – 2520030408, Keerthi - 2520030416** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. K. Anusha**, and my guide **Dr. K. Anusha**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

U. MOKSHA 2500031613

JAMUNA 2520030408

KEERTHI 2520030416

Abstract

The **Linux System Monitor** is a multithreaded Linux application developed using **C, POSIX threads, and mutex synchronization**. The main purpose of this project is to monitor important system resources such as **CPU usage, memory usage, disk usage, network activity, system uptime, and running processes** in a Linux environment.

The application uses multiple **POSIX threads** to perform different monitoring tasks concurrently. Each thread is responsible for collecting specific system information, while a **mutex** is used to protect shared data from simultaneous access by multiple threads. This demonstrates important Operating System concepts such as **processes, threads, concurrency, shared resources, critical sections, mutual exclusion, and synchronization**.

The project also includes a **race-condition demonstration**, where a shared counter is updated both without a mutex and with mutex protection. The results clearly show how unsynchronized access can produce incorrect values and how mutex synchronization prevents this problem. Linux interfaces such as the **/proc filesystem** are used to obtain system information, and the application is compiled using **GCC and Make**.

Overall, the project provides a practical demonstration of **multithreading and synchronization in Linux** and shows how Operating System concepts can be applied to develop a real-time system monitoring application.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–5
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		6–12
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–22
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		23–29
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	

		5.5 Input and Output Design	
6	Implementation		30–45
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	Execution and Demonstration		62–67
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	Conclusion		79–83
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	References		84–86
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of project

Modern computer systems run many processes and applications simultaneously, making it important to monitor system resources efficiently. Linux provides several system interfaces, such as the **/proc filesystem**, that allow applications to obtain information about CPU usage, memory, processes, network activity, and system uptime.

The **Linux System Monitor** project is developed to make use of these Linux facilities through a **C-based multithreaded application**. Instead of collecting all information sequentially, the project uses multiple **POSIX threads**, where each thread is responsible for monitoring a particular system resource. This allows different monitoring tasks to execute concurrently.

Since multiple threads access shared monitoring data, synchronization is required to prevent inconsistent results. A **POSIX mutex** is therefore used to protect shared resources and critical sections. The project also demonstrates a **race condition** by comparing shared-data access with and without mutex protection.

Thus, the project provides practical implementation of important Operating System concepts such as **processes, threads, concurrency, shared resources, critical sections, mutexes, mutual exclusion, and race conditions** in a Linux environment.

1.2 Problem Statement

Modern computer systems run many processes and applications simultaneously, making it important to monitor system resources efficiently. Linux provides several system interfaces, such as the **/proc filesystem**, that allow applications to obtain information about CPU usage, memory, processes, network activity, and system uptime.

The **Linux System Monitor** project is developed to make use of these Linux facilities through a **C-based multithreaded application**. Instead of collecting all information sequentially, the project uses multiple **POSIX threads**, where each thread is responsible for monitoring a particular system resource. This allows different monitoring tasks to execute concurrently.

Since multiple threads access shared monitoring data, synchronization is required to prevent inconsistent results. A **POSIX mutex** is therefore used to protect shared resources and critical sections. The project also demonstrates a **race condition** by comparing shared-data access with and without mutex protection.

Thus, the project provides practical implementation of important Operating System concepts such as **processes, threads, concurrency, shared resources, critical sections, mutexes, mutual exclusion, and race conditions** in a Linux environment.

1.3 Scope of the project

The scope of the **Linux System Monitor** project is to develop a Linux-based application that can monitor

important system resources using **C, POSIX threads, and mutex synchronization**.

The project covers the following areas:

- **CPU Monitoring:** Collecting and displaying CPU usage.
- **Memory Monitoring:** Monitoring the amount of memory being used by the system.
- **Disk Monitoring:** Checking disk space usage.
- **Network Monitoring:** Monitoring received and transmitted network data.
- **Process Monitoring:** Identifying running processes and displaying process IDs.
- **System Uptime:** Displaying the amount of time the Linux system has been running.
- **Multithreading:** Using separate POSIX threads to monitor different resources concurrently.
- **Mutex Synchronization:** Protecting shared data using mutex locks to avoid conflicts between threads.
- **Race-Condition Demonstration:** Showing the difference between unsynchronized and synchronized access to shared resources.
- **Linux System Interfaces:** Using Linux `/proc` files and system interfaces to obtain real-time system information.

1.4 Objectives of the Project

The main objectives of the **Linux System Monitor** project are:

- To develop a Linux-based system monitoring application using **C programming**.
- To monitor **CPU, memory, disk, network, system uptime, and running processes**.
- To use **POSIX threads** for concurrent monitoring of different system resources.
- To implement **mutex synchronization** for safe access to shared data.
- To demonstrate the concept of **race conditions** and show how mutexes prevent them.
- To provide practical understanding of **processes, threads, concurrency, critical sections, and mutual exclusion**.
- To obtain system information using Linux interfaces such as the **`/proc` filesystem**.

1.5 Importance of Application

The Linux System Monitor is important because it provides a practical way to understand how an operating system manages and monitors system resources. The application demonstrates how multiple tasks can execute concurrently using POSIX threads while maintaining data consistency through mutex synchronization.

The project also helps students understand the practical effects of **race conditions** and the importance of protecting shared resources in multithreaded programs. It connects theoretical Operating System concepts with an actual Linux-based application.

1.6 Target Users/Audience

The target users of this project include:

- **Operating Systems students** who want to understand processes and threads.
- **Linux beginners** interested in system monitoring.
- **C programmers** learning POSIX thread programming.

- **Students learning synchronization and mutexes.**
- **Academic users** who need a practical demonstration of concurrency and race conditions.

CHAPTER 2 — OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

The Linux System Monitor demonstrates several important Operating System concepts, including:

Processes

A process is a program in execution. The project identifies running processes by examining numeric

directories in the Linux `/proc` filesystem.

Threads

A thread is a lightweight unit of execution within a process. The project uses multiple POSIX threads to monitor different system resources concurrently.

Concurrency

CPU, memory, disk, network, and process monitoring tasks are performed by separate threads, allowing multiple monitoring operations to execute concurrently.

Shared Resources

The monitoring threads update shared variables containing CPU usage, memory usage, disk usage, network information, and other system data.

Critical Section

The portion of code where shared data is accessed or modified is treated as a critical section.

Mutex Synchronization

A POSIX mutex is used to ensure that only one thread accesses the protected shared data at a time.

Race Condition

A race condition occurs when multiple threads access and modify shared data without proper synchronization. The project demonstrates this problem using a shared counter.

Mutual Exclusion

Mutex locking provides mutual exclusion by allowing only one thread to execute the protected critical section at a time.

2.2 Existing Methods / Algorithms

Traditional Linux system monitoring is commonly performed using built-in command-line utilities. Commands such as `top`, `ps`, `free`, `df`, and network-monitoring utilities provide information about running processes and system resources.

Although these commands are useful, they generally operate as separate utilities and do not demonstrate the internal implementation of **multithreading and synchronization** in a C application.

The existing approach can be summarized as:

1. Execute a Linux monitoring command.
2. Read the displayed system information.

3. Repeat the command whenever updated information is required.
4. Analyze the output manually.

The main limitation of this approach is that it does not provide a single C application demonstrating concurrent resource monitoring with **POSIX threads and mutex synchronization**.

2.3 Proposed Approach

The proposed **Linux System Monitor** is a multithreaded C application that monitors multiple system resources concurrently.

The application creates separate POSIX threads for different monitoring tasks:

- **CPU Thread** – monitors CPU usage.
- **Memory Thread** – monitors memory usage.
- **Disk Thread** – monitors disk usage.
- **Process Thread** – monitors running processes and process IDs.
- **Network Thread** – monitors network data received and transmitted.

The collected information is stored in shared variables. A **POSIX mutex** is used whenever the shared data is accessed or modified.

The proposed approach works as follows:

1. Start the Linux System Monitor.
2. Initialize the shared variables and mutex.
3. Create the monitoring threads.
4. Each thread collects information from the Linux system.
5. The thread locks the mutex before updating shared data.
6. The shared data is updated safely.
7. The mutex is unlocked.
8. The main program reads and displays the collected information.
9. The monitoring process continues until the application is terminated.

The project also contains a **race-condition demonstration** to compare shared-data access without synchronization and with mutex protection.

2.4 Advantages of the Proposed Approach

The proposed approach provides the following advantages:

- **Concurrent Monitoring:** Multiple resources can be monitored simultaneously using separate threads.
- **Better Understanding of Threads:** Demonstrates practical use of POSIX threads in Linux.
- **Safe Data Access:** Mutex synchronization protects shared resources from conflicting access.
- **Race-Condition Demonstration:** Clearly shows the effect of accessing shared data without synchronization.
- **Real-Time System Information:** The application obtains current information directly from Linux system interfaces.
- **Single Application:** Different system-monitoring tasks are integrated into one application.
- **Practical OS Learning:** Provides hands-on understanding of processes, threads, concurrency, and synchronization.

- **Linux Integration:** Uses Linux interfaces such as the `/proc` filesystem to obtain system information.

CHAPTER 3 — SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The Linux System Monitor does not require high-end hardware because it is a lightweight C-based application. The basic hardware requirements are:

- **Processor:** Dual-core processor or higher
- **RAM:** Minimum 4 GB
- **Storage:** At least 1 GB of available space
- **Display:** Standard monitor
- **Keyboard:** Required for terminal interaction
- **Network:** Required only when monitoring network activity

The application can run on a normal personal computer or laptop capable of running Ubuntu/Linux.

3.2 Software Requirements

The following software is required to develop and execute the project:

Software	Purpose
Ubuntu/Linux	Operating system and execution environment
GCC	Compiling C source code
GNU Make	Automating compilation
POSIX Threads	Creating and managing threads
Linux Terminal	Running commands and the application
/proc Filesystem	Obtaining system information

The application mainly uses standard C libraries and Linux/POSIX interfaces.

3.3 Operating System Environment – Ubuntu / WSL

The project is designed to run in a **Linux environment**, particularly Ubuntu. Ubuntu provides the required POSIX threading support and Linux system interfaces used by the application.

The project can also be developed using **WSL (Windows Subsystem for Linux)** when a native Ubuntu installation is not available.

The Linux `/proc` filesystem is an important part of the project. It provides information about:

- CPU statistics through `/proc/stat`
- Memory information through `/proc/meminfo`
- Network statistics through `/proc/net/dev`
- System uptime through `/proc/uptime`
- Running processes through `/proc/<PID>`

3.4 Development Tools and Compilers

GCC Compiler

GCC (GNU Compiler Collection) is used to compile the C source files of the project. The pthread library is linked during compilation to provide POSIX thread functionality.

Example:

```
gcc src/main.c src/system_resources.c src/process_monitor.c src/race_demo.c -o monitor -pthread
```

GNU Make

A **Makefile** is used to simplify the compilation process. Instead of entering the complete GCC command manually, the project can be compiled using:

```
make
```

Linux Terminal

The terminal is used to compile, execute, and test the application. Commands such as `make`, `./monitor`, `ps`, and `ls` are useful during development and demonstration.

CHAPTER 4 — TECHNOLOGY AND TOOLS USED

4.1 C Programming

The project is implemented using the **C programming language**. C is suitable for this project because it provides efficient system-level programming and direct interaction with Linux/POSIX interfaces.

C is used for:

- Creating POSIX threads
- Managing shared variables
- Implementing mutex synchronization
- Reading Linux system information
- Processing process information
- Displaying monitoring results

The project is divided into multiple source and header files to keep the implementation organized and modular.

4.2 GCC Compiler

GCC (GNU Compiler Collection) is used to compile the C source files of the Linux System Monitor project. It converts the C source code into an executable program that can be executed in the Linux environment.

The project uses the `-pthread` option to enable POSIX thread support.

Example compilation command:

```
gcc src/main.c src/system_resources.c src/process_monitor.c src/race_demo.c -o monitor -pthread
```

The GCC compiler also helps identify errors and warnings during development. The project Makefile simplifies the compilation process by storing the required compiler options and source files.

4.3 Linux System Calls / Interfaces

The project uses Linux system interfaces to obtain information about the running system. The **/proc filesystem** is particularly important because it provides real-time information maintained by the Linux kernel.

The major interfaces used are:

/proc/stat

Used by the CPU monitoring thread to obtain CPU time statistics. These values are compared over a time interval to calculate CPU utilization.

/proc/meminfo

Used by the memory monitoring thread to obtain information about total and available/free memory.

/proc/net/dev

Used by the network monitoring thread to obtain the number of bytes received and transmitted through network interfaces.

/proc/uptime

Used to obtain the amount of time the system has been running.

/proc

The process-monitoring thread scans numeric directories inside `/proc`. Each numeric directory represents a process ID (PID).

statvfs()

The `statvfs()` function is used to obtain filesystem information such as total and available disk space.

These Linux interfaces allow the application to collect system information without relying on external monitoring applications.

4.4 POSIX Threads / Synchronization

POSIX threads, commonly called **pthread**s, are used to perform multiple monitoring tasks concurrently.

The project creates separate threads for:

1. CPU monitoring
2. Memory monitoring
3. Disk monitoring
4. Process monitoring
5. Network monitoring

The main functions used for thread management are:

- `pthread_create()` – creates a new thread.
- `pthread_join()` – waits for a thread to finish.
- `pthread_mutex_lock()` – locks the shared resource.
- `pthread_mutex_unlock()` – releases the shared resource.

A mutex called `data_mutex` is used to protect shared monitoring variables.

For example:

```
pthread_mutex_lock(&data_mutex);  
  
cpu_usage = usage;  
  
pthread_mutex_unlock(&data_mutex);
```

This ensures that another thread cannot modify the protected data simultaneously.

4.5 Linux Commands and Utilities

Several Linux commands are useful for compiling, executing, and testing the project.

make

Compiles the complete project using the Makefile.

`make`

./monitor

Runs the Linux System Monitor.

```
./monitor
```

make clean

Removes the previously generated executable.

```
make clean
```

ps

Displays information about running processes.

```
ps
```

ps -T

Displays threads associated with a process.

```
ps -T -p <PID>
```

top

Provides a standard Linux view of running processes and resource usage.

```
top
```

These commands are useful for testing and verifying the results produced by the project.

4.6 Other Tools / Libraries Used

The project uses several standard C and POSIX libraries.

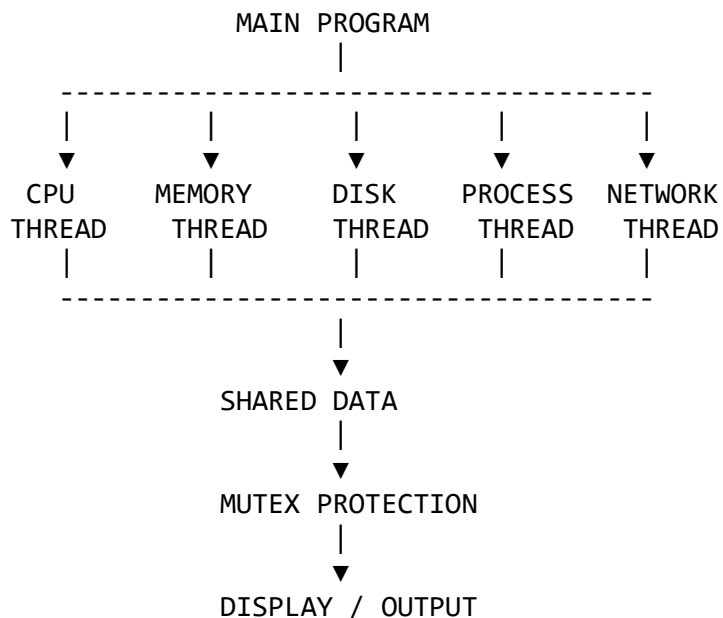
Library	Purpose
stdio.h	File handling and input/output
pthread.h	POSIX thread and mutex operations
unistd.h	Sleep and Linux/POSIX functions
dirent.h	Reading process directories
ctype.h	Checking numeric characters
stdlib.h	General utilities and conversions
string.h	String processing
sys/statvfs.h	Disk/filesystem information

The combination of these libraries provides the functionality required to collect system information and implement concurrent monitoring.

CHAPTER 5 — SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The Linux System Monitor follows a **multithreaded architecture**. The main program creates five monitoring threads, with each thread responsible for a particular system resource.



The architecture allows the monitoring operations to execute concurrently while the mutex provides safe access to shared data.

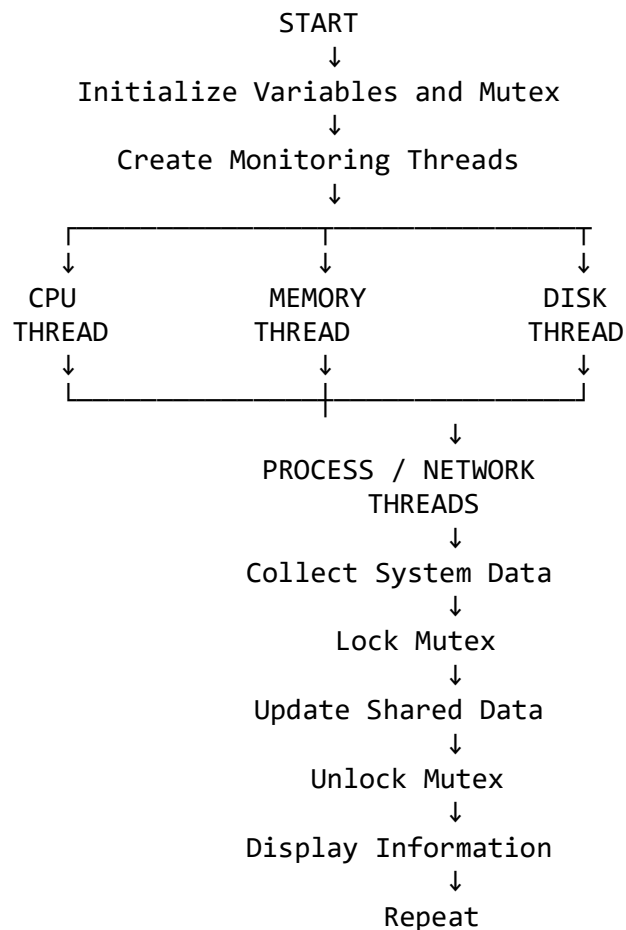
5.2 System Workflow

The system follows these steps:

1. The application starts.
2. Shared variables and the mutex are initialized.
3. Five POSIX monitoring threads are created.
4. Each thread collects information from its assigned Linux interface.
5. The thread locks the mutex before updating shared data.
6. The required shared variable is updated.
7. The mutex is unlocked.
8. The main program reads the latest values.
9. The monitoring information is displayed.
10. The process continues until the application is terminated.

This workflow allows the system monitor to continuously collect and display updated information.

5.3 Flowchart / Data Flow



5.4 Module Description

The project is divided into the following modules:

CPU Monitoring Module

Reads `/proc/stat` and calculates CPU utilization.

Memory Monitoring Module

Reads `/proc/meminfo` and calculates memory usage.

Disk Monitoring Module

Uses `statvfs()` to determine disk space utilization.

Process Monitoring Module

Scans `/proc` and identifies running process IDs.

Network Monitoring Module

Reads `/proc/net/dev` to obtain received and transmitted byte counts.

Race Condition Module

Creates multiple threads that modify a shared counter. The module demonstrates the difference between execution without a mutex and execution with mutex protection.

Main Module

Creates the monitoring threads, reads shared data, and displays the system-monitoring output.

5.5 Input and Output Design

The primary input to the application is the **Linux system environment itself**. The application reads information from Linux system interfaces and processes it.

Inputs

- CPU statistics
- Memory statistics
- Disk information
- Network statistics
- Process information
- System uptime

Processing

The monitoring threads collect and process the information concurrently. Mutex synchronization is used whenever shared monitoring data is accessed.

Outputs

The application displays:

- CPU usage
- Memory usage
- Disk usage
- Network information
- System uptime
- Number of running processes
- Process IDs
- Thread information
- Mutex status

The output is continuously updated while the monitoring application is running.

CHAPTER 6 — IMPLEMENTATION

6.1 Module-wise Implementation

The Linux System Monitor is divided into different modules so that each part of the application performs a specific task. This makes the program easier to understand, develop, test, and maintain.

1. Main Module

The main module is responsible for starting the application and creating the required POSIX threads. It also coordinates the monitoring process and displays the collected information.

2. CPU Monitoring Module

The CPU thread reads CPU statistics from `/proc/stat`. It compares CPU values collected at two different time intervals and calculates the CPU utilization percentage.

3. Memory Monitoring Module

The memory thread reads information from `/proc/meminfo`. It obtains the total and available/free memory and calculates the memory utilization.

4. Disk Monitoring Module

The disk thread uses the `statvfs()` function to obtain filesystem information. It calculates the percentage of disk space currently being used.

5. Process Monitoring Module

The process thread scans the `/proc` directory and identifies directories containing numeric names. These directories represent running process IDs.

6. Network Monitoring Module

The network thread reads `/proc/net/dev` and collects the number of bytes received and transmitted through network interfaces.

7. Race Condition Demonstration Module

This module creates two threads that update a common counter. The counter is first modified without synchronization and then with mutex protection. The results demonstrate why synchronization is necessary.

6.2 Algorithm / Pseudocode

Main Algorithm

START

Initialize shared variables

Initialize mutex

Create CPU monitoring thread

Create Memory monitoring thread

Create Disk monitoring thread

Create Process monitoring thread

Create Network monitoring thread

WHILE application is running

 Update system uptime

 Lock mutex

 Read CPU usage

 Read memory usage

 Read disk usage

 Read network information

 Read process information

 Unlock mutex

 Display system information

END WHILE

STOP

CPU Monitoring Algorithm

Read CPU statistics from /proc/stat

Wait for a short time

Read CPU statistics again

Calculate total CPU time difference

Calculate idle time difference

Calculate CPU usage percentage

Lock mutex

Update CPU usage

Unlock mutex

Memory Monitoring Algorithm

Open /proc/meminfo

Read total memory

Read free/available memory

Calculate memory usage percentage

Lock mutex

Update memory usage

Unlock mutex

Process Monitoring Algorithm

Open /proc directory

FOR every directory entry

 Check whether the directory name is numeric

 IF numeric

 Treat it as a process ID

 Store the process ID

 Increase process count

END FOR

Close /proc directory

Lock mutex

Update process information

Unlock mutex

6.3 Core C Implementation

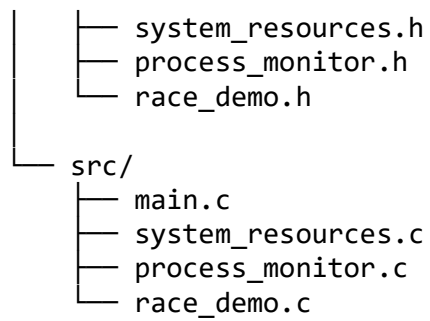
The project is implemented using multiple C source files and header files.

The main source files are:

linux-system-monitor/

├── Makefile

└── include/



The header files contain declarations of shared variables, functions, thread functions, and constants.

The source files contain the actual implementation of the monitoring modules.

The program uses `pthread_create()` to create the monitoring threads and `pthread_mutex_lock()` / `pthread_mutex_unlock()` to protect shared data.

6.4 Linux APIs Used

The following Linux/POSIX APIs and functions are used in the project:

API / Function	Purpose
<code>pthread_create()</code>	Creates a new thread
<code>pthread_join()</code>	Waits for a thread to complete
<code>pthread_mutex_lock()</code>	Locks a mutex
<code>pthread_mutex_unlock()</code>	Unlocks a mutex
<code>fopen()</code>	Opens Linux system files
<code>fscanf()</code>	Reads formatted data
<code>fgets()</code>	Reads lines from files
<code>opendir()</code>	Opens a directory
<code>readdir()</code>	Reads directory entries
<code>closedir()</code>	Closes a directory
<code>statvfs()</code>	Obtains filesystem information
<code>sleep()</code>	Controls monitoring intervals

These functions provide the required functionality for Linux system monitoring and multithreaded execution

6.5 Process / Thread Implementation

The project uses **five monitoring threads**:

Thread	Responsibility
CPU Thread	CPU usage
Memory Thread	Memory usage
Disk Thread	Disk usage

Process Thread	Running processes and PIDs
Network Thread	Network activity

Each thread runs independently and repeatedly performs its assigned monitoring task.

For example, the CPU thread continuously reads `/proc/stat`, while the memory thread reads `/proc/meminfo`. Since these tasks are independent, using separate threads allows them to execute concurrently.

The main program coordinates the threads and displays their collected results.

6.6 IPC / Synchronization

The project uses a **POSIX mutex** as its primary synchronization mechanism.

The monitoring threads share variables such as:

```
cpu_usage
memory_usage
disk_usage
network_received
network_transmitted
system_uptime
process_count
```

Since these variables are shared between threads, simultaneous access could cause inconsistent results.

Therefore, the project follows this pattern:

```
Lock Mutex
  ↓
Access / Modify Shared Data
  ↓
Unlock Mutex
```

For example:

```
pthread_mutex_lock(&data_mutex);

cpu_usage = usage;

pthread_mutex_unlock(&data_mutex);
```

This ensures that only one thread accesses the protected shared data at a time.

6.7 Memory Management

The project mainly uses local variables and standard C data structures for processing system information.

Files opened for reading are properly closed after the required information is collected. Similarly, the

/proc directory is closed after process information has been read.

The project does not require complex dynamic memory allocation for its core monitoring functions. This keeps the application lightweight and reduces the possibility of memory-management errors.

6.8 Optional User Interface

The core project operates through the Linux terminal and displays the collected monitoring information in a structured format.

An optional web interface can also be used to present the collected information in a browser. A Flask-based backend can provide system data through an API, while HTML, CSS, and JavaScript can be used for presentation.

The web interface is an optional presentation layer; the main Operating Systems implementation remains the **C application using POSIX threads and mutex synchronization**.